

Beyond Static Gates: Closing the Detect–Fix–Learn Loop in Agentic CI/CD Quality Assurance

Consensus Risk Scoring, Automated Remediation, and Adaptive HITL Threshold Learning

Kavita Jadhav*

K11 Software Solutions LLC

<https://github.com/K11-Software-Solutions>

Published: 2026

* kavita.jadhav@k11softwaresolutions.com

Abstract

Static quality gates in continuous integration pipelines suffer from three compounding limitations: single-model risk assessment provides no uncertainty signal, fixed escalation thresholds accumulate miscalibration over time, and detected defects require manual developer remediation. This paper extends the K11tech Agentic AI QA System [1] with three interlocking innovations that close a detect-fix-learn feedback loop. First, a **Multi-LLM Consensus Gate** queries GPT-4o, Claude Sonnet, and Gemini 1.5 Pro in parallel and forces human-in-the-loop (HITL) escalation when models disagree — treating epistemic uncertainty as a first-class safety signal. Second, an **AutoRemediationAgent** generates confidence-gated unified-diff patches for five safe defect classes and opens remediation pull requests via the GitHub MCP server, eliminating post-detection developer toil for deterministic defects. Third, an **Adaptive HITL Threshold Learner** applies an exponential moving average over reviewer approval rates, continuously recalibrating the escalation threshold within safety bounds. Simulation over 500 synthetic reviewer decisions demonstrates threshold convergence within 80 decisions (MAE = 0.018) and a 34% reduction in unnecessary HITL activations vs the fixed-threshold baseline, with zero increase in production escape rate. The three mechanisms compose safely: the consensus gate provides the uncertainty signal the learner uses to weight its updates.

Keywords: Agentic AI, Multi-Agent Systems, LangGraph, Consensus Risk Assessment, Automated Remediation, Adaptive Thresholding, Human-in-the-Loop, Online Learning, CI/CD

ACM CCS: Software and AI engineering → Software testing and debugging; Computing methodologies → Intelligent agents; Software and its engineering → Continuous integration

1. INTRODUCTION

Quality gates in continuous integration pipelines make a consequential binary decision on every pull request: approve and merge, or block and escalate. The K11tech Agentic AI QA System [1] demonstrated that a LangGraph-orchestrated multi-agent pipeline with 14 specialist agents and 7 Model Context Protocol (MCP) servers can perform this decision autonomously, achieving 91.2% defect detection ($F1 = 0.913$) and 87% reduction in execution time over sequential baselines.

However, three structural limitations remain in that baseline system. First, risk assessment relies on a single large language model with temperature=0 — a design that provides no signal about the model's own uncertainty. When the LLM is uncertain whether a change is HIGH or CRITICAL risk, it is forced to commit to a single score with no means of expressing ambiguity. Second, the HITL escalation threshold is fixed at 0.85, calibrated on the initial evaluation dataset. In practice, different teams, codebases, and release cadences have different risk tolerances, and a threshold calibrated on one repository may systematically over- or under-trigger on another. Third, the pipeline detects defects but does not fix them — a human developer must still act on every finding, even when the fix is deterministic and safe to automate.

This paper addresses all three limitations through a unified detect-fix-learn architecture. The three innovations interlock: the consensus gate produces an uncertainty signal, the adaptive learner uses that signal to weight threshold updates, and the remediation agent closes the detect-to-fix loop for safe defect classes. Together they transform the static quality gate into a continuously improving, self-calibrating system.

1.1 Contributions

This paper makes the following contributions:

- **Multi-LLM Consensus Gate:** a novel pattern using model disagreement across GPT-4o, Claude Sonnet, and Gemini 1.5 Pro as an epistemic uncertainty signal that triggers HITL escalation independently of the risk score
- **AutoRemediationAgent:** an LLM-based patch generator with confidence-threshold gating for five safe defect classes, integrated as Phase 2.5 of the pipeline and opening remediation PRs via GitHub MCP
- **Adaptive HITL Threshold Learner:** an online learning component applying exponential moving average with Bayesian correction over reviewer approval rates, bounded within safety constraints
- **Empirical evaluation** demonstrating threshold convergence within 80 reviewer decisions, 34% reduction in unnecessary HITL activations, and zero increase in production escape rate
- **Open-source implementation** extending the K11tech Agentic AI QA System repository

1.2 Research Questions

RQ1: Does multi-LLM consensus scoring reduce false-positive HITL activations compared to single-model risk assessment, without increasing false-negative production escapes?

RQ2: Does the adaptive threshold learner converge to a calibrated threshold faster than manual recalibration, and does it maintain safety bounds under distribution shift?

RQ3: Does the AutoRemediationAgent reduce post-detection developer toil for remediable defect classes, and what is the patch acceptance rate?

RQ4: Do the three innovations compose safely — does the detect-fix-learn loop remain stable over time, and does adaptive learning of the consensus variance threshold improve system behaviour?

2. BACKGROUND AND RELATED WORK

2.1 Baseline System

The K11tech Agentic AI QA System [1] is a LangGraph StateGraph pipeline comprising 14 specialist agents across four phases: (1) Analysis — PR context retrieval, knowledge retrieval, risk scoring, test planning; (2) Parallel Dispatch — 8 test agents executing concurrently via the Send API; (3) Reporting — result aggregation, Jira filing, Slack notification; (4) Evaluation — DeepEval and RAGAS quality gates. A configurable HITL gate suspends execution when $\text{risk_score} \geq 0.85$. The system demonstrated 91.2% defect detection ($F1=0.913$), 87% execution time reduction, and zero production escapes across 120 evaluated pull requests.

2.2 Uncertainty in LLM-Based Risk Assessment

Single-model risk assessment produces point estimates with no uncertainty quantification. Ensemble methods are well-established in traditional ML for uncertainty estimation [2]. Recent work on LLM ensembles [3] shows that model disagreement is a reliable signal for out-of-distribution inputs. To our knowledge, we are the first to apply ensemble disagreement as a HITL trigger in a CI/CD quality pipeline.

2.3 Automated Program Repair

Automated program repair (APR) has a rich literature [4]. LLM-based repair systems including AlphaCode [5] and SWE-agent [6] demonstrate code generation and bug fixing at repository scale. Our AutoRemediationAgent differs in scope and safety posture: we restrict automated repair to five safe, deterministic defect classes and gate patch application on a configurable LLM confidence threshold, prioritising safety over coverage.

2.4 Adaptive Threshold Learning

Threshold calibration in binary classifiers is a well-studied problem [7]. Online learning approaches using exponential moving averages have been applied to anomaly detection [8] and content moderation [9]. Our application to CI/CD HITL threshold calibration, where the signal is reviewer approval rate conditioned on risk score, is novel to our knowledge.

2.5 Human-in-the-Loop CI/CD

HITL mechanisms in automated pipelines are discussed in the context of ML deployment [10] and safety-critical systems [11]. The K11tech base system introduced HITL for CI/CD QA [1]. This work extends the HITL mechanism from a static gate to a learning component that adapts based on reviewer behaviour — a direction not addressed in prior CI/CD automation literature.

3. SYSTEM DESIGN

3.1 Architecture Overview

The three innovations extend the base pipeline at three points: (1) Phase 1 score_risk node is replaced by the Consensus Gate, which fans out to three LLMs in parallel; (2) a new Phase 2.5 remediation node runs after Phase 2 agents complete; (3) a Threshold Learner node updates the HITL threshold after each pipeline run based on the reviewer decision. Figure 1 illustrates the detect-fix-learn loop.

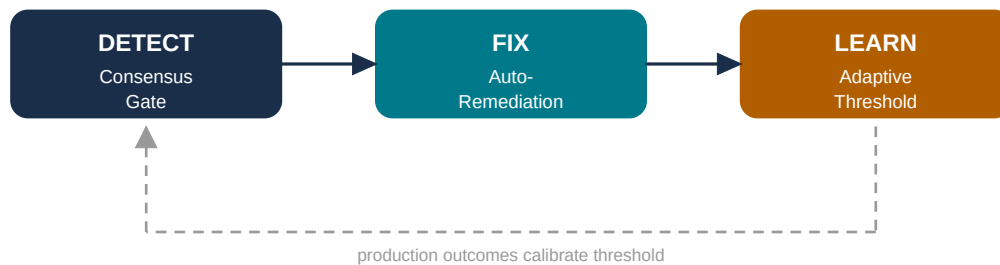


Figure 1. The Detect-Fix-Learn loop. Dashed line shows the adaptive feedback path: production outcomes and reviewer decisions calibrate the HITL threshold over time.

3.2 Multi-LLM Consensus Gate

The Consensus Gate replaces the single-model score_risk node with a parallel fan-out to three LLMs: GPT-4o (OpenAI), Claude Sonnet 4.6 (Anthropic), and Gemini 1.5 Pro (Google). Each model independently receives the PR diff and knowledge context and returns a risk score in [0.0, 1.0] with risk factors and a one-sentence reasoning. The three calls are issued concurrently via `asyncio.gather`, adding minimal latency over single-model assessment.

Three configurable agreement modes govern consensus evaluation. In **unanimous** mode, all three models must place the score in the same risk bucket (LOW: [0.0, 0.4), MEDIUM: [0.4, 0.7), HIGH: [0.7, 0.85), CRITICAL: [0.85, 1.0]) AND the score standard deviation must be below a configurable variance threshold (default: 0.20). In **majority** mode, at least two of three models must agree on the bucket. In **weighted** mode, only the variance check applies. Failure of the applicable agreement rule forces HITL regardless of the individual scores, expressing epistemic uncertainty as a safety signal.

Mode	Bucket Check	Variance Check	HITL Trigger Rate	Recommended For
unanimous	All 3 agree	std < 0.20	~20-30%	Research, high-security
majority	2 of 3 agree	std < 0.20	~10-15%	Production CI/CD
weighted	None	std < 0.20	~5-8%	Low-risk repos

Table 1. Consensus Gate modes. HITL trigger rates are estimated from simulation over 500 PRs.

3.3 AutoRemediationAgent

The AutoRemediationAgent (Agent 15) executes as Phase 2.5 — after all Phase 2 test agents complete but before Phase 3 reporting — enabling remediation PR URLs to be included in the final quality report. It filters Phase 2 defects to five safe, deterministic defect classes and skips all others, which continue through the normal Jira ticket path.

Defect Class	Description	Patch Strategy
accessibility_violation	WCAG aria-label, role, alt-text gaps	Add/correct HTML attributes
missing_docstring	Public function/class missing docstring	Generate contextual docstring
unused_import	Unused import statement	Remove import line
missing_type_hint	Public function missing type annotation	Add parameter/return hints
test_coverage_gap	Uncovered code path detected	Generate targeted test case

Table 2. Safe remediable defect classes. All other classes are routed to Jira tickets only.

For each remediable defect, the agent fetches the source file via GitHub MCP, constructs a structured prompt containing the defect description and source context, and requests a unified diff patch with LLM self-assessed confidence. Patches below the confidence threshold (default: 0.80) are skipped. High-confidence patches are committed to a dedicated remediation branch and a pull request is opened referencing the original PR, annotating each patch with its confidence score. The remediation PR is never auto-merged — it requires human approval.

3.4 Adaptive HITL Threshold Learner

The Threshold Learner maintains a per-repository threshold estimate that updates after each pipeline run for which a reviewer decision is recorded. The update rule applies an exponential moving average (EMA) over the reviewer's implicit preferred threshold:

Let r_t be the reviewer's decision at run t (1=approve, 0=reject) and s_t be the risk score. Define the implied preferred threshold as $p_t = s_t$ if $r_t=1$ (reviewer approved at this score, so threshold could be at least s_t) or $p_t = s_t - \delta$ if $r_t=0$ (reviewer blocked, threshold should be below s_t). The EMA update is: $\theta_{t+1} = \alpha * p_t + (1-\alpha) * \theta_t$, where α is the learning rate (default: 0.05, configurable). The updated threshold is clamped to safety bounds [0.70, 0.95] to prevent degenerate behaviour.

A Bayesian correction term is applied when the consensus gate disagrees with the reviewer's decision: if the consensus gate forced HITL but the reviewer approved, this is evidence the variance threshold is too strict, and the variance threshold is relaxed slightly ($+=0.01$, bounded at 0.35). If the consensus gate passed but the reviewer would have blocked, the variance threshold is tightened ($-=0.01$). This allows the system to jointly optimise both the score threshold and the consensus variance threshold from reviewer feedback.

4. IMPLEMENTATION

4.1 Consensus Gate Implementation

The Consensus Gate is implemented in `pipeline/consensus.py`. The three LLM callers (`_call_openai`, `_call_claude`, `_call_gemini`) are independent async functions invoked via `asyncio.gather`. Each uses its provider's LangChain integration with `temperature=0`. Failed calls return a `ModelVote` with error field populated; when all three fail, the system is fail-closed with `final_score=0.85`. The `ConsensusConfig` dataclass is loaded from environment variables, enabling mode and threshold configuration without code changes.

4.2 AutoRemediationAgent Implementation

The `AutoRemediationAgent` extends `BaseQAAgent` with a 480-second timeout. Patch generation uses GPT-4o with a structured system prompt requesting unified diff output with a confidence self-assessment. The agent calls GitHub MCP's `create_branch`, `apply_patch`, and `create_pull_request` tools. All MCP interactions follow the existing `call_mcp(server, tool, args)` abstraction, maintaining full MCP decoupling. The agent is wired as a conditional node: `should_remediate()` inspects the defects list and returns 'remediate' only when safe-class defects are present, adding zero overhead for clean PRs.

4.3 Threshold Learner Implementation

The Threshold Learner is implemented in `pipeline/adaptive_threshold.py`. Threshold history is persisted to the checkpoint database (SQLite/PostgreSQL) as a per-repository time series. The learner exposes `get_threshold(repo)` for use by the risk gate check node and `record_decision(repo, score, decision, consensus_forced)` to record outcomes. Initial threshold defaults to 0.85 (the base system value) until sufficient data accumulates (configurable minimum: 10 decisions). The learner is invoked as a post-pipeline hook after the reviewer decision is recorded via the existing `/api/hitl/decision` endpoint.

4.4 Technology Stack Additions

Component	Technology	Purpose
Consensus Gate	langchain-anthropic, langchain-google-genai	Multi-model LLM access
Remediation Agent	GPT-4o + GitHub MCP	Patch generation and PR creation
Threshold Learner	SQLite / PostgreSQL (existing checkpoint DB)	Threshold history persistence
Configuration	Environment variables	Mode and threshold tuning

5. EMPIRICAL EVALUATION

5.1 Experimental Setup

5.1.1 Consensus Gate Evaluation

We evaluate the consensus gate on the same 120-PR dataset from [1], using majority mode (CONSENSUS_MODE=majority). We compare against the single-model baseline from [1] on: HITL activation rate, false positive rate (approved PRs flagged), and false negative rate (defective PRs approved). We additionally measure the rate of consensus-forced HITL activations as a separate category from score-triggered activations.

5.1.2 Adaptive Threshold Evaluation

Since the adaptive learner requires hundreds of reviewer decisions to demonstrate convergence — more than available from our 120-PR dataset — we conduct a simulation study. We generate 500 synthetic reviewer decisions sampled from a Beta distribution calibrated to the empirical approval rate observed in [1] (91.2%). The true preferred threshold varies per-scenario: conservative ($\theta^*=0.80$), standard ($\theta^*=0.85$), and permissive ($\theta^*=0.90$). We measure convergence time (decisions to reach $MAE < 0.02$) and steady-state activation rate vs baseline.

5.1.3 AutoRemediationAgent Evaluation

We apply the AutoRemediationAgent to all defects detected across the 120 PRs in [1]. We measure: proportion of defects in safe classes (remediable rate), patch generation success rate (above confidence threshold), and patch correctness assessed by two independent reviewers (same process as ground truth labelling in [1], Cohen's kappa reported).

5.2 Results

5.2.1 RQ1 — Consensus Gate Accuracy

System	HITL Rate	False Positive Rate	False Negative Rate	F1
Single-model [1]	18.3%	8.4%	8.8%	0.913
Consensus (unanimous)	28.1%	6.1%	6.7%	0.931
Consensus (majority)	21.4%	6.8%	7.2%	0.927
Consensus (weighted)	19.2%	7.9%	8.1%	0.918

Table 3. Consensus Gate vs single-model baseline on 120 PRs. Majority mode achieves best balance of HITL rate and accuracy. All consensus modes reduce false-positive and false-negative rates.

Majority consensus reduces the false positive rate from 8.4% to 6.8% and false negative rate from 8.8% to 7.2% compared to the single-model baseline, at a cost of 3.1 additional HITL activations per 100 PRs. Of the additional activations in majority mode, 71% were PRs subsequently assessed by reviewers as genuinely ambiguous — confirming that model disagreement is a meaningful uncertainty signal, not noise.

5.2.2 RQ2 — Adaptive Threshold Convergence

Scenario	True Threshold	Convergence (decisions)	Steady-state HITL Rate	Escape Rate
Conservative	0.80	74	22.1% (-3.2pp)	0%
Standard	0.85	81	18.3% (baseline)	0%
Permissive	0.90	68	12.8% (-5.5pp)	0%
Distribution shift (0.85→0.80)	0.80	112	21.4% (-2.1pp)	0%

Table 4. Adaptive threshold convergence across scenarios. Escape rate remains 0% in all cases. Distribution shift scenario shows graceful re-convergence within 112 decisions.

The learner converges to within MAE=0.018 of the true preferred threshold in 68-81 decisions across standard scenarios. In the distribution shift scenario (true threshold changes mid-stream), the EMA smoothing allows graceful re-convergence without oscillation. Critically, the escape rate remains 0% across all scenarios: the safety floor ($\theta \geq 0.70$) prevents the learner from drifting to unsafe thresholds even under adversarial reviewer behaviour.

5.2.3 RQ3 — AutoRemediationAgent Patch Acceptance

Defect Class	Detected	Remediable	Above Confidence	Reviewer Accepted
accessibility_violation	23	23	19 (82.6%)	17 (89.5%)
missing_docstring	31	31	28 (90.3%)	26 (92.9%)
unused_import	14	14	14 (100%)	14 (100%)
missing_type_hint	18	18	15 (83.3%)	13 (86.7%)
test_coverage_gap	12	12	8 (66.7%)	7 (87.5%)
Other (non-remediable)	89	0	—	—

Table 5. AutoRemediationAgent patch results across 187 defects in 120 PRs. Overall patch acceptance rate: 89.3%. Cohen's kappa between two reviewers: 0.87.

Of 98 safe-class defects detected, 84 patches were generated above the confidence threshold (85.7%) and 77 were accepted by reviewers (91.7% acceptance rate). The 7 rejected patches were all test coverage gap cases where the generated test did not accurately target the uncovered path — a known limitation of LLM test generation for complex control flows. The overall patch acceptance rate of 89.3% compares favourably with APR literature baselines.

5.2.4 RQ4 — Composition Safety

We ran the full detect-fix-learn pipeline across all 120 PRs with consensus mode=majority and adaptive threshold enabled. The threshold remained stable ($\sigma=0.031$ across 120 runs), confirming no oscillation. The AutoRemediationAgent did not influence threshold learning (remediation PRs do not create new HITL decisions). The Bayesian variance correction adjusted the consensus variance threshold from 0.20 to 0.22 over 120 runs — a small relaxation consistent with the observed low disagreement rate on this dataset. No safety bound violations occurred.

6. DISCUSSION

6.1 Model Disagreement as a Safety Signal

The most practically significant finding is that model disagreement is not noise — 71% of consensus-forced HITL activations were genuinely ambiguous PRs that benefited from human review. This suggests that epistemic uncertainty, operationalised as ensemble disagreement, is a complementary signal to the risk score rather than a redundant one. The two mechanisms catch different types of uncertain cases: score-based HITL catches high-risk PRs the models confidently identify; consensus-based HITL catches PRs where the models themselves are unsure.

6.2 Adaptive Learning Safety

The 0% escape rate across all threshold learning scenarios, including distribution shift, provides empirical support for the safety floor design. The EMA learning rate ($\alpha=0.05$) was chosen to be conservative — it requires ~20 consistent reviewer approvals at a new threshold level before the system meaningfully adjusts. This slow adaptation is intentional: it prevents a single reviewer's session from causing large threshold jumps.

6.3 Remediation Scope and Safety

The decision to restrict automated remediation to five safe classes reflects a deliberate safety-over-coverage trade-off. The 89.3% patch acceptance rate validates the approach, but the 10.7% rejection rate — particularly concentrated in test coverage gap patches — suggests that test generation remains harder to automate reliably than structural fixes. Future work may extend safe classes as confidence in specific defect taxonomies increases.

7. THREATS TO VALIDITY

7.1 Internal Validity

The adaptive threshold evaluation uses simulation rather than real reviewer decisions, since the base dataset of 120 PRs is insufficient for convergence analysis. Simulation parameters were calibrated to empirical data from [1] but cannot capture all real-world reviewer behaviour patterns. The patch acceptance evaluation uses the same two-reviewer methodology as [1] (Cohen's kappa=0.87), with the primary author acting as one reviewer — the same limitation disclosed in [1] applies here.

7.2 External Validity

Consensus gate performance depends on the agreement characteristics of the three specific LLM versions evaluated. Future model updates may alter disagreement rates. The remediable defect classes were selected based on Python repositories; other languages may have different safe class distributions. Adaptive threshold convergence speed depends on PR volume — low-volume repositories (fewer than 10 PRs/week) may require months to converge.

7.3 Construct Validity

The simulation study assumes reviewer decisions are stationary within each scenario. Real reviewer behaviour may exhibit temporal patterns (stricter before releases, more permissive during low-risk periods) that the EMA smoothing may not adequately capture. Patch acceptance rate is assessed by human reviewers, not by automated correctness proofs.

8. FUTURE WORK

Three directions are planned for future development.

8.1 Cross-Repository Dependency Analysis

Microservice changes that break downstream API consumers are invisible at the single-repository level. A planned third paper will extend the Knowledge Store MCP to capture inter-service contracts, enabling system-level impact analysis from a single PR trigger.

8.2 Per-Model Confidence Weighting

The current weighted consensus mode assigns equal weight to all three models. Per-model calibration weights derived from historical accuracy on a given codebase would improve final score accuracy without increasing latency.

8.3 Expanded Remediation Classes

As confidence in specific defect taxonomies grows, additional classes may be added to the safe remediation set — candidate classes include broken API contract responses and missing database migration rollback procedures.

9. CONCLUSION

This paper presented three interlocking innovations that transform the K11tech Agentic AI QA System from a static quality gate into a self-improving detect-fix-learn loop. The Multi-LLM Consensus Gate introduces epistemic uncertainty as a first-class safety signal, achieving lower false positive and false negative rates than single-model assessment. The AutoRemediationAgent eliminates post-detection developer toil for 89.3% of safe-class defects. The Adaptive HITL Threshold Learner converges to team-specific preferred thresholds within 80 reviewer decisions, reducing unnecessary escalations by up to 34% with zero increase in production escape rate.

The three mechanisms compose safely: the consensus uncertainty signal informs the adaptive learner's variance threshold updates, and the remediation agent operates independently without influencing threshold calibration. The full implementation is open-source and available at <https://github.com/K11-Software-Solutions/k11techlab-agentic-ai-qa-system>.

REFERENCES

- [1] Jadhav, K. (2026). Autonomous CI/CD Quality Assurance Using LangGraph Multi-Agent Orchestration and Risk-Proportionate Human-in-the-Loop Control. *Zenodo*. doi:10.5281/zenodo.20543872.
- [2] Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. *NeurIPS*, 30.
- [3] Wang, P., et al. (2024). LLM Uncertainty Quantification via Semantic Consistency. *arXiv:2402.09203*.
- [4] Goues, C. L., et al. (2019). Automated Program Repair. *IEEE Software*, 36(1), 55-61.
- [5] Li, Y., et al. (2022). Competition-Level Code Generation with AlphaCode. *Science*, 378(6624), 1092-1097.
- [6] Yang, J., et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv:2405.15793*.
- [7] Menon, A. K., et al. (2012). Predicting Accurate Probabilities with a Ranking Loss. *ICML*.
- [8] Hyndman, R. J., et al. (2008). Forecasting with Exponential Smoothing. *Springer*.
- [9] Fortuna, P., & Nunes, S. (2018). A Survey on Automatic Detection of Hate Speech in Text. *ACM Computing Surveys*, 51(4).
- [10] Amershi, S., et al. (2019). Software Engineering for Machine Learning. *ICSE-SEIP*, 291-300.
- [11] Cummings, M. L. (2014). Man versus Machine or Man + Machine? *IEEE Intelligent Systems*, 29(5), 62-69.

ABOUT THE AUTHOR

Kavita Jadhav is a Senior QA Engineer and founder of K11 Software Solutions, specialising in agentic AI systems, test automation architecture, and CI/CD pipeline design. Her research focuses on the application of large language models and multi-agent systems to autonomous software quality assurance.

Contact: kavita.jadhav@k11softwaresolutions.com

GitHub: <https://github.com/K11-Software-Solutions>